How the recommended sets are found

The logic behind the Set Search panel's "Recommended Sets": the function `recommend` in `frontend/src/features/builder/recommend.ts`. Line numbers and measurements are as of commit `9351ab7`, with the Master Rank armor loaded (1,670 armor pieces, 314 charms, 404 decorations, 130 skills).

---

1. The short version

You list the skills you want, most important first, each with a target level. The panel answers with up to ten complete builds: five armor pieces, a charm, and the jewels already placed. All of it comes from one pure function, `recommend(wanted, catalog, limit = 10)`.

Three ideas carry it:

1. **"Best" follows your list order.** A build is judged on the first skill, then the second, and so on. Defense only breaks ties.
2. **Jewels never need searching.** Putting them in by priority, each into the smallest slot it fits, is always the best way to fill a given set of slots.
3. **Most of the catalog is never tried.** Pieces that another piece beats outright are dropped before the search, and a half-built set is abandoned the moment it can no longer reach the top ten.

It is an exact search, not a heuristic: within the rules listed in section 8, no build in the catalog beats the first result.

---

2. What "best" means

For every wanted skill a build has the level it reaches, never counted past the target. Two builds are compared skill by skill, in list order. The first skill where they differ decides. If they are level on every skill, the higher defense wins.

Say the list is Attack Boost 7, Critical Eye 7, Weakness Exploit 3:

Build	Attack Boost (7)	Critical Eye (7)	Weakness Exploit (3)	Defense	Place
A	7	4	0	700	1st
C	7	4	0	650	2nd
B	6	7	3	900	3rd

B has more skill points in total and more defense than the other two, and still comes last: it is one level short on the first skill, and nothing lower in the list can make up for that. A and C are level on every skill, so defense separates them.

Two things follow from this rule:

- **The order of the list is the main control.** If a result looks lopsided, the list asked for it. Move a skill up, or lower a target above it.
- **Overshoot is worth nothing.** Attack Boost 9 from armor counts as 7, so the search never spends a slot on a skill that is already at its target.

Inside the solver the levels become the digits of one number, first skill leftmost: A is `740` and B is `673`, written in base 8 (one more than the highest target). Comparing two builds is then a single number comparison. This number is called the **score** below.

---

3. The pipeline at a glance

```
wanted skills, in priority order, each with a target level
        |
        v
4.1  clean the targets    drop unknown skills, hold levels to the cap, 12 at most
4.2  pick the jewels      per skill: the smallest jewel that grants only that skill
4.3  build the rows       every charm and piece -> points, usable slots, defense
4.5  prune, then sort     drop every row that another row beats or equals,
                          strongest first inside each group
4.6  look ahead           the most the groups still to come could add
4.7  search               charm, head, chest, arms, waist, legs: keep the best ten,
                          skip every branch that cannot reach them
4.8  place the jewels     on the real slots of the ten winners
        |
        v
ten ready builds, best first
```

The numbers are the sections below. Section 4.4 is the helper `fill`, which answers "with these points and these slots, what score can be reached?". The sort and the search both call it, and 4.8 repeats its rule on the real slots.

---

4. The steps

4.1 Clean the targets (lines 23-29)

The wanted list is saved in the browser and can be older than the catalog. Skills the catalog no longer has are dropped, each level is held between 1 and the skill's `max_level`, and only the first 12 (`MAX_WANTED`) are searched. An empty list returns nothing.

4.2 Pick one jewel per skill (lines 44-51)

For each wanted skill the solver chooses one decoration: the one with the smallest slot size among those that grant that skill and nothing else. Attack Boost gets Attack Jewel 1, not Attack Jewel+ 4. Earplugs gets Earplug Jewel 3.

25 of the 130 skills have no such jewel. Those can only come from armor and the charm.

4.3 Turn every charm and piece into a row (lines 9-16, 31-39, 55-61, 114-127)

There are six groups: charms, then head, chest, arms, waist, legs. Each item is reduced to the only three things that matter for this list:

- **points** toward each wanted skill. Skills that are not on the list are ignored.
- **slots**, counted by size, but only as far as the list can use them. A slot too small for every wanted jewel is not counted. A slot bigger than the biggest wanted jewel is counted as that size, because the extra room buys nothing.
- **defense**.

For the six-skill list of section 5 (the biggest wanted jewel is size 3):

Item	Points (AB, CE, WE, CB, HB, EP)	Real slots	Slots as counted	Defense
Attack Charm V	5, 0, 0, 0, 0, 0	none	none	0
Dragonbarbs Alpha +	0, 3, 1, 0, 0, 0	4, 3, 3	3, 3, 3	180
Frostfang Helm Alpha +	0, 2, 0, 0, 0, 0	3, 2, 1	3, 2, 1	162

Each group also gets a **blank** row in front: "wear nothing here".

Stripping items down like this is what makes so many of them equal or directly comparable, and that is what the pruning in 4.5 lives on.

4.4 Score a pile of points and slots: `fill` (lines 68-84)

Given total points and total slots, the jewels go in by priority. For the first skill, take the smallest slots its jewel fits until the target is reached or the slots run out. Then the second skill with what is left, and so on. The result is the score from section 2.

This is not an approximation, for two reasons:

- A jewel fits every slot of its size or bigger. Taking the smallest slot that fits therefore never uses up anything a later jewel could have used better.
- The first skill outranks everything after it, so giving it all it needs first is always right.

So the jewels need no search of their own, and a build is fully described by its six rows. A two-slot case from the tests: slots of size 3 and 1, wanted a size-1 jewel first and a size-3 jewel second. The first jewel takes the size-1 slot and leaves the size-3 slot to the only jewel that needs it.

4.5 Prune: drop the pieces that cannot win (lines 87-127)

Row Y makes row X pointless when Y has:

- at least X's points in every wanted skill,
- at least X's defense, and
- slots that **cover** X's: at every size, Y has at least as many slots of that size or bigger.

Any build that uses X is at least as good with Y swapped in, so X is dropped. When two rows are exactly equal the earlier one stays. The catalog arrives sorted by name, so that is Alpha before Beta.

Real cases from the six-skill list:

- **Escadora Wisdom Alpha +** (Critical Eye +2, slots 3 and 1, defense 160) is dropped. **Frostfang Helm Alpha +** has Critical Eye +2, slots 3, 2 and 1, and defense 162.
- **Frostfang Helm Beta +** (slots 4, 2, 1) is dropped as an equal twin of Alpha + (slots 3, 2, 1): with no wanted jewel bigger than size 3, its level 4 slot counts as a 3.
- "Covers" is not "has more slots". Slots 3 and 1 against slots 2 and 2: the first has a big slot the second lacks, the second has two slots of size 2 or more where the first has one. Neither covers the other and both stay.

What is dropped depends on the list. Escadora Wisdom Alpha + also carries Resentment +3. Put Resentment on the list and it survives; in section 6 it is in the winning set.

The blank row goes first so that it wins ties. A charm with none of the wanted skills is equal to wearing no charm, so it is dropped rather than one being picked at random. In the armor groups any piece with defense beats the blank, which only survives in a slot the catalog has no pieces for.

What pruning does to the two lists used in this document:

Group	In the catalog	Left, six skills	Left, twelve skills
Charm	314	6	12
Head	352	14	30
Chest	331	19	36
Arms	331	17	26
Waist	326	16	24
Legs	330	14	26
Combinations	1.3 × 10^15	6,077,568	210,263,040

The six charms left for six skills are simply the best charm of each skill.

The rows that are left are then sorted strongest first inside each group (line 129), by the score each would reach on its own. Good builds turn up early in the search, which gives the next step something to cut with.

4.6 Look ahead: two bounds (lines 131-148)

210 million combinations is still too many to score one by one. So before searching, the solver works out `ahead[d]` for every position `d` in the group order: the most the groups from `d` on could still add. It is deliberately optimistic. It must never be less than what a real choice gives, or the search would skip a build that deserved a place.

It takes each remaining group's best slots and best defense, and counts skill points in two different ways:

- **The per-skill bound** takes the best of every skill at once, as if one piece of each group carried that group's highest Attack Boost and its highest Critical Eye and so on.
- **The loose bound** keeps to the points the single most useful piece of each group really carries, but lets them go to any skill.

No real choice beats either one. Each is far too generous in a different situation, which is why the search uses both and stops as soon as either says no. Counting how many times the search function is entered:

Wanted list	Both bounds	Per-skill only	Loose only
Six skills (section 5)	6,942	30,158	6,942
Twelve damage skills at cap (section 6)	403,707	131,352,125	403,707
Twelve utility skills, most of them level 1	31,607	48,252	1,398,437

Damage skills sit on a great many pieces, so the imaginary piece that has all of them at once is far better than any real one, and the per-skill bound stops cutting: 0.2 seconds becomes 31. Scarce skills are the other way round: the loose bound pretends that points of a common skill could be spent on a rare one. Over 300 random lists, the per-skill bound cut branches the loose bound had missed on 174 of them.

4.7 The search (lines 150-186)

A depth-first walk: choose a charm, then a head piece, chest, arms, waist and legs, adding up points, slots and defense on the way down. With all six chosen, `fill` gives the score and the build goes into a list of the ten best so far, ordered by score and then defense.

Once that list holds ten, every step first asks: taking the most optimistic view of what is still to come, could this partial build beat the tenth entry? If either bound says no (a lower score, or the same score with no more defense), the whole branch is skipped without trying a single piece below it.

For the six-skill list: of 6,077,568 combinations the search function is entered 6,942 times and 171 complete builds are scored.

4.8 Place the jewels for real (lines 188-213)

The search only counted slots. For each of the ten winners the solver now takes the real slots of the chosen pieces, smallest first, and repeats the rule of 4.4: by priority, each jewel into the smallest free slot it fits, stopping at the target. Slots nobody needs stay empty.

What comes back is a `Build`, the same shape as the builder's own state, so choosing a recommended set is one `setBuild`.

---

5. Worked example: six skills

Every figure here is what `recommend` returned on the current catalog.

Wanted: Attack Boost 7, Critical Eye 7, Weakness Exploit 3, Critical Boost 3, Health Boost 3, Earplugs 5.

Jewels picked: Attack Jewel 1, Expert Jewel 1, Tenderizer Jewel 2, Critical Jewel 2, Vitality Jewel 1, Earplug Jewel 3. Smallest size 1, biggest size 3: every slot counts, and a level 4 slot counts as a 3.

The first result, defense 896, every target met:

Slot	Piece	Wanted skills it grants	Jewel in each slot, slot level in parentheses
Head	Dragonhead Beta +	none	(4) Expert 1, (4) Critical 2, (4) Vitality 1
Chest	Dragonhide Alpha +	Weakness Exploit +1	(4) Vitality 1, (4) Vitality 1, (3) Expert 1
Arms	Dragonclaws Beta +	Weakness Exploit +1	(4) Earplug 3, (4) Earplug 3, (4) Earplug 3
Waist	Dragonbarbs Alpha +	Critical Eye +3, Weakness Exploit +1	(4) Earplug 3, (3) Expert 1, (3) Expert 1
Legs	Kulve Taroth's Wrath Beta +	Critical Boost +2	(4) Earplug 3, (1) Attack 1, (1) Attack 1
Charm	Attack Charm V	Attack Boost +5	

Skill	From armor and charm	From jewels	Reached
Attack Boost	5	2	7 of 7
Critical Eye	3	4	7 of 7
Weakness Exploit	3	0	3 of 3
Critical Boost	2	1	3 of 3
Health Boost	0	3	3 of 3
Earplugs	0	5	5 of 5

Fifteen jewels in fifteen slots, with nothing to spare. The placement shows the rule of 4.4 at work. Attack Boost is first in the list and its jewel is size 1, so its two jewels took the two level 1 slots. The four Expert jewels took the three level 3 slots, then one level 4. Earplugs is last in the list and the only skill whose jewel needs a size 3 slot, and five level 4 slots were still free for it.

Because every target is met, the score is as high as it can go and defense decided the rest: four pieces at 180 and one at 176.

The other nine. The second result is the same five pieces with Earplugs Charm V in place of Attack Charm V, and the jewels rearranged to match: same score, same defense. Results three to ten change the head or the legs, the charm, or both, at defense 890, 890, 886, 886, 886, 886, 884, 884. The list ranks; it does not try to show ten different ideas.

---

6. When the list asks too much: twelve skills

Wanted, all at their cap: Attack Boost 7, Critical Eye 7, Weakness Exploit 3, Critical Boost 3, Agitator 5, Peak Performance 3, Maximum Might 3, Resentment 5, Handicraft 5, Fortify 1, Heroics 5, Latent Power 5.

The first result, defense 846: Escadora Wisdom Alpha +, Grand God's Peer Garb Beta +, Escadora Armguards Beta +, Dragonbarbs Alpha +, Kulve Taroth's Wrath Alpha +, Challenger Charm V.

Skills 1 to 7	Resentment	Handicraft	Fortify	Heroics	Latent Power
all at target	4 of 5	0 of 5	0 of 1	0 of 5	0 of 5

The build is spent strictly from the top of the list down, and the tail gets nothing. It is also a guarantee, under the rules of section 8: no build in the catalog reaches Resentment 5 while holding the seven skills above it at their targets.

Move Handicraft to the top and the answer is a different set altogether (Leon's Head Alpha +, Buff Chest Alpha +, Dragonclaws Alpha +, Dragonbarbs Alpha +, Rimeguard Greaves Gamma +, Attack Charm V, defense 844): Handicraft 5 of 5, the next six skills still at target, then Maximum Might 1 of 3, Resentment 0 of 5, Fortify 1 of 1, and nothing for the last two.

The panel announces this case with "No set reaches every target. Closest first." and names, on each set, the first two skills it falls short on.

---

7. How sure is the answer

- **The first result is the best build there is**, under the rules of section 8. Pruning cannot lose it (the piece that beat a dropped piece can always take its place), the bounds cannot lose it (they only skip branches that provably stay out of the list), and the jewel fill is exact.
- **Results two to ten are the next best among the pieces that survived pruning.** A build that differs from a listed one only by a beaten or twin piece is never shown, even where it would score the same. For the six-skill list, Frostfang Helm Beta + can never appear.
- **It is tested against brute force.** `recommend.test.ts`, run by `pnpm test`, builds 40 small random catalogs, tries every combination of pieces and charm with every way to fill the slots, and requires the solver's first result to match the best of them in both score and defense. Half the runs ask for a single result, which makes the bounds cut their hardest. The levels are counted by `computeTotals`, code the solver does not share. The test does not pin results two to ten, or the speed on the real catalog.

---

8. What it does not consider

- **Set bonuses.** They play no part in the ranking.
- **Level 4 jewels.** Combination jewels and "+" jewels are never used, so a level 4 slot gets a smaller jewel and the rest of it is wasted. 432 of the 863 pieces of rarity 9 and up have a level 4 slot, so a hand-made build can reach more than the first result does.
- **Secret skill caps.** Targets stop at the skill's `max_level`.
- **What you own.** Every piece, charm and jewel in the catalog is assumed to be available, in any quantity.
- **Resistances.** Only defense breaks a tie; elemental resistances are not looked at.
- **Variety.** Near-twins fill the list, as section 5 shows.

Cost. The search runs on the main thread on every change to the list. Measured with Node 25 on an Apple M4:

Wanted list	Time
300 random lists of 1 to 12 skills, median	1 ms
the same, 90th and 99th percentile	33 ms, 258 ms
Six skills (section 5)	4 ms
Twelve damage skills at cap (section 6)	0.19 s
Slowest found: twelve cheap skills, all met	1.0 s

The slowest one is not the list the comment above `MAX_WANTED` names (the damage list, at about 0.3 s). It is Resuscitate 1, Windproof 5, Hunger Resistance 1, Scoutfly Range Up 1, Capacity Boost 1, BBQ Master 1, Attack Boost 2, Blight Resistance 2, Weakness Exploit 1, Scenthound 1, Free Elem/Ammo Up 3, Resentment 4. Every one of its ten results meets every target, at defense 652 to 702 where five pieces could reach 900. That fits the code: when the candidates all tie on skills, only the defense half of the bound is left to cut with, and it assumes the toughest piece in every slot. In development React's StrictMode runs the search twice.

---

9. What the panel does with it

In `frontend/src/features/builder/components/set-search.tsx`:

- The wanted list is `wantedAtom`, saved in localStorage under `wanted`. A new skill is added at its cap. The up and down buttons change the order, which is the priority.
- The search reruns on every change, behind `useDeferredValue`: the list repaints first and the results follow. The first render searches an empty list, so the page paints before any search.
- Each build goes through `computeTotals`, the same function that feeds the Status panel, for its defense and for the targets it misses. The solver's own score is never shown.
- Choosing a set calls `setBuild` with it and replaces the current build. The set equal to the current build wears the green selection band.

10. Code map

All in `frontend/src/features/builder/recommend.ts`.

Part	Name	Lines
Most skills one search takes	`MAX_WANTED`	7
A charm or piece, reduced	`Row`	9-16
Entry point	`recommend`	21
Clean the targets	`targets`	23-29
Points toward the wanted skills	`points`	31-39
One jewel per skill	`jewels`	44-51
Slots worth counting	`usable`	55-61
Score from points and slots	`fill`	68-84
One set of slots covers another	`covers`	87-95
Drop beaten rows	`prune`	99-110
The six groups, blank first	`blank`, `groups`	114-127
Strongest first	sort	129
The two bounds	`ahead`	131-148
Depth-first search, best ten	`search`, `kept`	150-186
Jewels onto real slots	return value	188-213

The test is `recommend.test.ts` beside it, and the panel is `components/set-search.tsx`.